Pehchaan: source code

UnivaBio 2026 submission. Muhammad Taqui. MIT licence.

All code written for the project, in the order a reader would follow it: the web app, the risk engine, the interface text, the training pipeline, the tests and the documentation build script. Generated files (model.json, model.js, parity_vectors.json) and the dataset are not reprinted; they are produced by ml/train.py.

Contents

1. app/index.html (164 lines)
2. app/styles.css (156 lines)
3. app/app.js (304 lines)
4. app/risk.js (94 lines)
5. app/i18n.js (164 lines)
6. ml/train.py (236 lines)
7. tests/test_risk.js (69 lines)
8. scripts/build_docs.py (147 lines)
9. package.json (12 lines)
10. requirements.txt (5 lines)

1. app/index.html

```
<!doctype html>
<html lang="en" dir="ltr" data-theme="dark">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Pehchaan</title>
  <meta name="description" content="Early diabetes risk screening without a lab test. Explainable, bilingual (English and Urdu), runs fully in the browser.">
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="stylesheet" href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&family=JetBrains+Mono:wght@500&family=Noto+Nastaliq+Urdu:wght@400;600&display=swap">
  <link rel="stylesheet" href="styles.css">
  <link rel="icon" href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 32 32'%3E%3Crect width='32' height='32' rx='8' fill='%230B1F3B'/%3E%3Ccircle cx='16' cy='16' r='7' fill='none' stroke='%2300C2FF' stroke-width='3'/%3E%3C/svg%3E">
</head>
<body>
  <header class="topbar">
    <div class="wrap topbar-inner">
      <div class="brand">
        <svg class="brand-mark" viewBox="0 0 32 32" aria-hidden="true"><rect width="32" height="32" rx="8"/><circle cx="16" cy="16" r="7"/></svg>
        <span data-i18n="app_name">Pehchaan</span>
      </div>
      <div class="topbar-actions">
        <button id="langBtn" class="btn ghost" type="button" data-i18n="lang_toggle">اردو</button>
        <button id="themeBtn" class="btn ghost" type="button" aria-label="Toggle light or dark theme">
          <span data-i18n="theme_toggle">Theme</span>
        </button>
      </div>
    </div>
  </header>

  <main class="wrap">
    <section class="hero">
      <h1 data-i18n="tagline">Early diabetes risk screening, no lab test needed.</h1>
      <p class="lead" data-i18n="hero_body"></p>
      <p class="notice"><strong data-i18n="disclaimer"></strong> <span data-i18n="privacy"></span></p>
    </section>

    <div class="grid">
      <!-- Step 1: questionnaire -->
      <section class="card" id="formCard">
        <h2 data-i18n="form_title">About you</h2>
        <form id="riskForm" novalidate>
          <fieldset class="field">
            <legend data-i18n="sex">Sex</legend>
            <div class="seg">
              <label><input type="radio" name="sex" value="female" checked><span data-i18n="female">Female</span></label>
              <label><input type="radio" name="sex" value="male"><span data-i18n="male">Male</span></label>
            </div>
          </fieldset>

          <div class="row">
            <label class="field"><span data-i18n="age">Age</span>
              <input id="age" type="number" inputmode="numeric" min="18" max="110" required></label>
            <label class="field" id="pregField"><span data-i18n="pregnancies">Pregnancies</span>
              <input id="preg" type="number" inputmode="numeric" min="0" max="20" value="0"></label>
          </div>

          <div class="row">
            <label class="field"><span data-i18n="height">Height (cm)</span>
              <input id="height" type="number" inputmode="decimal" min="120" max="220" required></label>
            <label class="field"><span data-i18n="weight">Weight (kg)</span>
              <input id="weight" type="number" inputmode="decimal" min="30" max="250" required></label>
          </div>
          <p class="hint"><span data-i18n="bmi_label">Body mass index</span>: <output id="bmiOut" class="mono">—</output></p>

          <div class="field">
            <label for="bp" data-i18n="bp">Diastolic blood pressure</label>
```

```
            <div class="inline">
              <input id="bp" type="number" inputmode="numeric" min="40" max="140">
              <label class="check"><input id="bpUnknown" type="checkbox"><span data-i18n="unknown">I don't
know</span></label>
            </div>
          </div>

          <fieldset class="field">
            <legend data-i18n="family">Family history</legend>
            <div class="seg three">
              <label><input type="radio" name="family" value="none" checked><span data-
i18n="fam_none">None</span></label>
              <label><input type="radio" name="family" value="one"><span data-i18n="fam_one">One</span>
</label>
              <label><input type="radio" name="family" value="multiple"><span data-i18n="fam_multiple">Two or
more</span></label>
            </div>
          </fieldset>

          <p id="formError" class="error" role="alert" hidden></p>
          <div class="actions">
            <button class="btn primary" type="submit" data-i18n="submit">Check my risk</button>
            <button class="btn ghost" type="reset" data-i18n="reset">Start over</button>
          </div>
        </form>
      </section>

      <!-- Step 2: result and explanation -->
      <section class="card" id="resultCard" hidden aria-live="polite">
        <h2 data-i18n="result_title">Your result</h2>
        <div id="tier" class="tier"></div>
        <p id="tierMsg"></p>

        <div class="gauge" aria-hidden="true">
          <div class="gauge-fill" id="gaugeFill"></div>
          <div class="gauge-thr" id="gaugeThr"><span data-i18n="threshold_lbl">Threshold</span></div>
        </div>
        <p class="hint"><span data-i18n="score">Model score</span> <b id="scoreOut" class="mono"></b> · <span
id="pctOut"></span></p>
        <ul id="warnings" class="warnings"></ul>

        <h3 data-i18n="why_title">What drove this result</h3>
        <p class="hint" data-i18n="why_note"></p>
        <div id="bars" class="bars"></div>

        <h3 data-i18n="whatif_title">What if my weight changed?</h3>
        <input id="whatIf" type="range" step="1">
        <p id="whatIfOut" class="mono-ish"></p>
        <p class="hint" data-i18n="whatif_note"></p>

        <button id="printBtn" class="btn ghost" type="button" data-i18n="print">Print or save
summary</button>
      </section>
    </div>

    <div class="grid">
      <!-- Step 3: rule-based check of an existing lab result -->
      <section class="card">
        <h2 data-i18n="reading_title">Already have a blood sugar result?</h2>
        <p class="hint" data-i18n="reading_note"></p>
        <form id="readingForm" novalidate>
          <label class="field"><span data-i18n="reading_type">Test type</span>
            <select id="readingType">
              <option value="fasting" data-i18n="t_fasting">Fasting</option>
              <option value="ogtt" data-i18n="t_ogtt">OGTT</option>
              <option value="hba1c" data-i18n="t_hba1c">HbA1c</option>
              <option value="random" data-i18n="t_random">Random</option>
            </select></label>
          <div class="row">
            <label class="field"><span data-i18n="reading_value">Result</span>
              <input id="readingValue" type="number" inputmode="decimal" step="0.1" required></label>
            <label class="field" id="unitField"><span data-i18n="unit">Unit</span>
              <select id="readingUnit"><option value="mgdl">mg/dL</option><option
value="mmol">mmol/L</option></select></label>
          </div>
          <label class="check" id="symptomField" hidden><input id="symptoms" type="checkbox"><span data-
```

```
i18n="symptoms"></span></label>
          <div class="actions"><button class="btn primary" type="submit" data-i18n="check_reading">Check
result</button></div>
          <p id="readingOut" class="reading-out" aria-live="polite"></p>
        </form>
      </section>

      <!-- Model card -->
      <section class="card">
        <h2 data-i18n="how_title">How it works</h2>
        <p id="howBody"></p>
        <img src="evaluation.png" alt="ROC curve and calibration plot of the model on the held-out test set"
class="figure">
        <h3 data-i18n="limits_title">Known limitations</h3>
        <ul class="limits">
          <li id="lim1"></li><li id="lim2"></li><li id="lim3"></li><li id="lim4"></li>
        </ul>
      </section>
    </div>

    <!-- Only visible when printing -->
    <section id="printSummary" class="print-only"></section>
  </main>

  <footer class="wrap footer"><p data-i18n="footer"></p></footer>

  <script src="model.js"></script>
  <script src="risk.js"></script>
  <script src="i18n.js"></script>
  <script src="app.js"></script>
</body>
</html>
```

2. app/styles.css

```
/* Pehchaan styles. Dark theme first; 8px spacing scale. */
:root {
  --navy: #0B1F3B;
  --cyan: #00C2FF;
  --bg: #08162B;
  --surface: #0E2442;
  --border: #1E3A5F;
  --text: #E8EEF6;
  --muted: #9AB0C8;
  --neg: #5F7894;
  --warn: #FFC857;
  --high: #FF7A7A;
  --ok: #4FD1A5;
  --s1: 8px; --s2: 16px; --s3: 24px; --s4: 32px; --s6: 48px;
  --radius: 12px;
  --font: "Inter", system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
  --mono: "JetBrains Mono", ui-monospace, SFMono-Regular, Menlo, monospace;
  color-scheme: dark;
}
:root[data-theme="light"] {
  --bg: #F5F7FA;
  --surface: #FFFFFF;
  --border: #DCE3EC;
  --text: #111827;
  --muted: #5B6676;
  --neg: #A3B0C0;
  --warn: #8A5A00;
  --high: #C62828;
  --ok: #0F7B57;
  --cyan: #0089B8;
  color-scheme: light;
}

* { box-sizing: border-box; }
html, body { margin: 0; }
body {
  background: var(--bg);
  color: var(--text);
  font-family: var(--font);
  font-size: 16px;
  line-height: 1.55;
}
[dir="rtl"] body, [dir="rtl"] button, [dir="rtl"] input, [dir="rtl"] select {
  font-family: "Noto Nastaliq Urdu", var(--font);
}
[dir="rtl"] body { line-height: 2.1; }

.wrap { max-width: 1080px; margin: 0 auto; padding: 0 var(--s2); }
h1 { font-size: clamp(28px, 4vw, 40px); line-height: 1.2; margin: 0 0 var(--s2); letter-spacing: -0.01em; }
h2 { font-size: 20px; margin: 0 0 var(--s2); }
h3 { font-size: 15px; margin: var(--s3) 0 var(--s1); color: var(--muted); font-weight: 600; }
[dir="rtl"] h1, [dir="rtl"] h2, [dir="rtl"] h3 { line-height: 1.9; }

.topbar { border-bottom: 1px solid var(--border); background: var(--surface); position: sticky; top: 0; z-
index: 5; }
.topbar-inner { display: flex; align-items: center; justify-content: space-between; height: 64px; }
.brand { display: flex; align-items: center; gap: var(--s1); font-weight: 700; font-size: 18px; }
.brand-mark { width: 28px; height: 28px; }
.brand-mark rect { fill: var(--navy); stroke: var(--border); }
.brand-mark circle { fill: none; stroke: #00C2FF; stroke-width: 3; }
.topbar-actions { display: flex; gap: var(--s1); }

.hero { padding: var(--s6) 0 var(--s4); max-width: 760px; }
.lead { color: var(--muted); font-size: 18px; margin: 0 0 var(--s2); }
.notice { border-inline-start: 3px solid var(--cyan); padding: var(--s1) var(--s2); background: var(--
surface); border-radius: 0 var(--radius) var(--radius) 0; margin: 0; }
[dir="rtl"] .notice { border-radius: var(--radius) 0 0 var(--radius); }

.grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(320px, 1fr)); gap: var(--s3); margin-
bottom: var(--s3); align-items: start; }
.card { background: var(--surface); border: 1px solid var(--border); border-radius: var(--radius); padding:
var(--s3); }

```

```
  70 .field { display: flex; flex-direction: column; gap: 6px; margin: 0 0 var(--s2); border: 0; padding: 0; min-
width: 0; }
  71 .field > span, .field > legend, .field > label { font-size: 14px; color: var(--muted); font-weight: 500;
padding: 0; }
  72 .row { display: grid; grid-template-columns: 1fr 1fr; gap: var(--s2); }
  73 input[type="number"], select {
  74   width: 100%; padding: 10px 12px; font-size: 16px; color: var(--text);
  75   background: var(--bg); border: 1px solid var(--border); border-radius: 8px;
  76 }
  77 input:focus-visible, select:focus-visible, button:focus-visible, .seg label:focus-within {
  78   outline: 2px solid var(--cyan); outline-offset: 2px;
  79 }
  80 input:disabled { opacity: 0.5; }
  81 .inline { display: flex; gap: var(--s2); align-items: center; }
  82 .inline input[type="number"] { flex: 1; }
  83 .check { display: flex; gap: var(--s1); align-items: center; font-size: 14px; color: var(--muted); white-
space: nowrap; }
  84 .check input { width: 18px; height: 18px; accent-color: var(--cyan); }
  85 #symptomField { white-space: normal; margin-bottom: var(--s2); align-items: flex-start; }
  86
  87 .seg { display: grid; grid-template-columns: 1fr 1fr; border: 1px solid var(--border); border-radius: 8px;
overflow: hidden; }
  88 .seg.three { grid-template-columns: 1fr 1fr 1fr; }
  89 .seg label { position: relative; text-align: center; cursor: pointer; }
  90 .seg input { position: absolute; opacity: 0; inset: 0; cursor: pointer; }
  91 .seg span { display: block; padding: 10px 8px; font-size: 14px; color: var(--muted); }
  92 .seg label + label { border-inline-start: 1px solid var(--border); }
  93 .seg input:checked + span { background: var(--navy); color: #fff; box-shadow: inset 0 -2px 0 #00C2FF; }
  94 :root[data-theme="light"] .seg input:checked + span { background: var(--navy); }
  95
  96 .hint { font-size: 14px; color: var(--muted); margin: 0 0 var(--s2); }
  97 .mono, .mono-ish { font-family: var(--mono); }
  98 .error { color: var(--high); font-size: 14px; margin: 0 0 var(--s2); }
  99
 100 .btn { font: inherit; font-size: 15px; font-weight: 600; border-radius: 8px; padding: 10px 16px; cursor:
pointer; border: 1px solid var(--border); }
 101 .btn.primary { background: #00C2FF; color: #04111F; border-color: #00C2FF; }
 102 .btn.primary:hover { filter: brightness(1.08); }
 103 .btn.ghost { background: transparent; color: var(--text); }
 104 .btn.ghost:hover { border-color: var(--cyan); }
 105 .actions { display: flex; gap: var(--s1); flex-wrap: wrap; }
 106
 107 .tier { display: inline-block; font-weight: 700; padding: 6px 12px; border-radius: 999px; font-size: 15px; }
 108 .tier.high { background: color-mix(in srgb, var(--high) 18%, transparent); color: var(--high); }
 109 .tier.low { background: color-mix(in srgb, var(--ok) 18%, transparent); color: var(--ok); }
 110
 111 .gauge { position: relative; height: 12px; background: var(--bg); border: 1px solid var(--border); border-
radius: 999px; margin: var(--s4) 0 var(--s1); direction: ltr; }
 112 .gauge-fill { position: absolute; inset: 0 auto 0 0; background: var(--cyan); border-radius: 999px;
transition: width .4s ease; }
 113 .gauge-thr { position: absolute; top: -8px; bottom: -8px; width: 2px; background: var(--text); }
 114 .gauge-thr span { position: absolute; bottom: 100%; left: 50%; transform: translateX(-50%); font-size: 11px;
color: var(--muted); white-space: nowrap; }
 115
 116 .warnings { list-style: none; padding: 0; margin: 0 0 var(--s2); }
 117 .warnings li { color: var(--warn); font-size: 14px; padding-inline-start: var(--s2); position: relative; }
 118 .warnings li::before { content: "!"; position: absolute; inset-inline-start: 0; font-weight: 700; }
 119
 120 .bars { display: grid; gap: 10px; }
 121 .bar-row { display: grid; grid-template-columns: 120px 1fr; gap: var(--s1); align-items: center; font-size:
14px; }
 122 .bar-track { position: relative; height: 14px; direction: ltr; }
 123 .bar-track::before { content: ""; position: absolute; left: 50%; top: -3px; bottom: -3px; width: 1px;
background: var(--border); }
 124 .bar { position: absolute; top: 0; bottom: 0; border-radius: 3px; }
 125 .bar.up { left: 50%; background: var(--cyan); }
 126 .bar.down { right: 50%; background: var(--neg); }
 127 .bar-note { grid-column: 2; font-size: 12px; color: var(--muted); margin-top: -6px; }
 128
 129 input[type="range"] { width: 100%; accent-color: var(--cyan); }
 130 .reading-out { margin: var(--s2) 0 0; font-weight: 500; }
 131 .reading-out.normal { color: var(--ok); }
 132 .reading-out.prediabetes, .reading-out.random_low, .reading-out.random_high { color: var(--warn); }
 133 .reading-out.diabetes, .reading-out.random_high_symptoms { color: var(--high); }
 134
 135 .figure { width: 100%; border-radius: 8px; background: #fff; margin: var(--s1) 0; }
```

```
.limits { padding-inline-start: 20px; margin: 0; font-size: 14px; color: var(--muted); }
.limits li { margin-bottom: var(--s1); }

.footer { color: var(--muted); font-size: 13px; padding-bottom: var(--s4); border-top: 1px solid var(--
border); padding-top: var(--s2); }
.print-only { display: none; }

@media (max-width: 520px) {
  .row { grid-template-columns: 1fr; gap: 0; }
  .bar-row { grid-template-columns: 96px 1fr; }
  .hero { padding-top: var(--s4); }
}

@media print {
  body { background: #fff; color: #000; }
  .topbar, .hero, .grid, .footer { display: none !important; }
  .print-only { display: block; font-family: var(--font); }
  .print-only h1 { font-size: 22px; }
  .print-only table { border-collapse: collapse; width: 100%; margin: 12px 0; }
  .print-only td { border: 1px solid #ccc; padding: 6px 8px; }
}
```

3. app/app.js

```
/* Pehchaan UI: reads the form, calls the risk engine, renders the result. */
(function () {
  "use strict";

  const MODEL = window.PEHCHAAN_MODEL;
  const RISK = window.PehchaanRisk;
  const I18N = window.PEHCHAAN_I18N;
  const $ = (id) => document.getElementById(id);

  let lang = "en";
  let lastInput = null; // last valid questionnaire answers, reused on language change

  // ---------- language ----------
  function t(key, vars) {
    let s = I18N[lang][key] ?? I18N.en[key] ?? key;
    if (vars) for (const [k, v] of Object.entries(vars)) s = s.split("{" + k + "}").join(v);
    return s;
  }

  function fmt(x, digits) {
    return Number(x).toFixed(digits);
  }

  function applyLanguage() {
    document.documentElement.lang = lang;
    document.documentElement.dir = I18N[lang].dir;
    document.querySelectorAll("[data-i18n]").forEach((el) => {
      el.textContent = t(el.dataset.i18n);
    });
    renderModelCard();
    if (lastInput) renderResult(lastInput);
    const rv = $("readingOut");
    if (rv.dataset.cat) rv.textContent = t("cat_" + rv.dataset.cat);
  }

  // ---------- model card ----------
  function renderModelCard() {
    const m = MODEL.test_metrics;
    $("howBody").textContent = t("how_body", {
      n: MODEL.dataset.n_total,
      auc: fmt(m.auc, 2),
      auc_lo: fmt(m.ci95.auc[0], 2),
      auc_hi: fmt(m.ci95.auc[1], 2),
      sens: fmt(m.sensitivity, 2),
      spec: fmt(m.specificity, 2),
      ref: fmt(MODEL.reference_auc_with_glucose, 2),
    });
    $("lim1").textContent = t("limit_1");
    $("lim2").textContent = t("limit_2", { prev: Math.round(MODEL.dataset.prevalence * 100) });
    $("lim3").textContent = t("limit_3");
    $("lim4").textContent = t("limit_4", { miss: Math.round((1 - m.sensitivity) * 100) });
  }

  // ---------- form ----------
  function num(id) {
    const v = $(id).value.trim();
    return v === "" ? NaN : Number(v);
  }

  function sex() {
    return document.querySelector('input[name="sex"]:checked').value;
  }

  function updateBmi() {
    const h = num("height"), w = num("weight");
    $("bmiOut").textContent = h > 0 && w > 0 ? fmt(RISK.bmi(h, w), 1) : "—";
  }

  function readForm() {
    const checks = [
      ["age", 18, 110], ["height", 120, 220], ["weight", 30, 250],
    ];
    if (sex() === "female") checks.push(["preg", 0, 20]);
```

```
  if (!$("bpUnknown").checked && $("bp").value.trim() !== "") checks.push(["bp", 40, 140]);

  for (const [id, a, b] of checks) {
    const v = num(id);
    if (!(v >= a && v <= b)) {
      if (id === "age" && v < 18 && v > 0) return { error: t("err_adult"), field: id };
      return { error: t("err_range", { a, b }), field: id };
    }
  }
  const family = document.querySelector('input[name="family"]:checked').value;
  return {
    sex: sex(),
    age: num("age"),
    height: num("height"),
    weight: num("weight"),
    pregnancies: sex() === "female" ? num("preg") : 0,
    bp: $("bpUnknown").checked || $("bp").value.trim() === "" ? null : num("bp"),
    family: family,
  };
}

/** Maps questionnaire answers onto the model's five features. */
function toFeatures(a, weightOverride) {
  return {
    Pregnancies: a.pregnancies,
    BloodPressure: a.bp,
    BMI: RISK.bmi(a.height, weightOverride ?? a.weight),
    DiabetesPedigreeFunction: MODEL.family_history_dpf[a.family],
    Age: a.age,
  };
}

function warningsFor(a, f) {
  const r = MODEL.training_ranges;
  const w = [];
  if (a.sex === "male") w.push(t("warn_male"));
  if (a.age < r.Age[0] || a.age > r.Age[1]) w.push(t("warn_age"));
  if (f.BMI < r.BMI[0] || f.BMI > r.BMI[1]) w.push(t("warn_bmi"));
  if (a.bp !== null && (a.bp < r.BloodPressure[0] || a.bp > r.BloodPressure[1])) w.push(t("warn_bp"));
  return w;
}

// ---------- result ----------
function renderResult(a) {
  const f = toFeatures(a);
  const res = RISK.score(MODEL, f);
  const high = res.aboveThreshold;

  const tier = $("tier");
  tier.className = "tier " + (high ? "high" : "low");
  tier.textContent = t(high ? "tier_high" : "tier_low");
  $("tierMsg").textContent = t(high ? "tier_high_msg" : "tier_low_msg");

  $("gaugeFill").style.width = (res.probability * 100).toFixed(1) + "%";
  $("gaugeThr").style.left = (MODEL.threshold * 100).toFixed(1) + "%";
  $("scoreOut").textContent = fmt(res.probability, 2);
  $("pctOut").textContent = t("percentile_tpl", { p: RISK.percentile(MODEL, res.probability) });

  const warn = $("warnings");
  warn.replaceChildren(...warningsFor(a, f).map((s) => {
    const li = document.createElement("li");
    li.textContent = s;
    return li;
  }));

  renderBars(res);
  setupWhatIf(a);
  buildPrintSummary(a, f, res);
}

function renderBars(res) {
  const entries = Object.entries(res.contributions).sort((x, y) => Math.abs(y[1]) - Math.abs(x[1]));
  const maxAbs = Math.max(0.5, ...entries.map(([, v]) => Math.abs(v)));
  const rows = entries.map(([feat, v]) => {
    const row = document.createElement("div");
    row.className = "bar-row";
```

```
      const label = document.createElement("span");
      label.textContent = t("feat_" + feat);
      const track = document.createElement("div");
      track.className = "bar-track";
      track.title = (v >= 0 ? "+" : "") + v.toFixed(2) + " log-odds";
      const bar = document.createElement("div");
      bar.className = "bar " + (v >= 0 ? "up" : "down");
      bar.style.width = ((Math.abs(v) / maxAbs) * 50).toFixed(1) + "%";
      track.appendChild(bar);
      row.append(label, track);
      if (res.imputed.includes(feat)) {
        const note = document.createElement("span");
        note.className = "bar-note";
        note.textContent = t("imputed");
        row.appendChild(note);
      }
      return row;
    });
    $("bars").replaceChildren(...rows);
  }

  function setupWhatIf(a) {
    const s = $("whatIf");
    const lo = Math.max(30, Math.round(a.weight - 20));
    const hi = Math.round(a.weight + 10);
    s.min = lo; s.max = hi;
    if (!s.dataset.touched || Number(s.value) < lo || Number(s.value) > hi) s.value = Math.round(a.weight -
5);
    updateWhatIf();
  }

  function updateWhatIf() {
    if (!lastInput) return;
    const w = Number($("whatIf").value);
    const f = toFeatures(lastInput, w);
    const p = RISK.score(MODEL, f).probability;
    $("whatIfOut").textContent = t("whatif_tpl", { w: w, b: fmt(f.BMI, 1), s: fmt(p, 2) });
  }

  function buildPrintSummary(a, f, res) {
    const root = $("printSummary");
    root.replaceChildren();
    const h = document.createElement("h1");
    h.textContent = t("summary_title");
    const p1 = document.createElement("p");
    p1.textContent = t(res.aboveThreshold ? "tier_high" : "tier_low") + ". " + t(res.aboveThreshold ?
"tier_high_msg" : "tier_low_msg");
    const table = document.createElement("table");
    const rows = [
      [t("sex"), t(a.sex)],
      [t("age"), a.age],
      [t("height"), a.height],
      [t("weight"), a.weight],
      [t("bmi_label"), fmt(f.BMI, 1)],
      [t("pregnancies"), a.sex === "female" ? a.pregnancies : "—"],
      [t("bp"), a.bp ?? t("unknown")],
      [t("family"), t("fam_" + a.family)],
      [t("score"), fmt(res.probability, 2) + " (" + t("threshold_lbl") + " " + MODEL.threshold + ")"],
    ];
    for (const [k, v] of rows) {
      const tr = table.insertRow();
      tr.insertCell().textContent = k;
      tr.insertCell().textContent = String(v);
    }
    const p2 = document.createElement("p");
    p2.textContent = t("disclaimer");
    const p3 = document.createElement("p");
    p3.textContent = new Date().toLocaleDateString(lang === "ur" ? "ur-PK" : "en-GB");
    root.append(h, p1, table, p2, p3);
  }

  // ---------- reading check ----------
  function updateReadingFields() {
    const type = $("readingType").value;
    $("unitField").hidden = type === "hba1c";
    $("symptomField").hidden = type !== "random";
```

```
  }

  // ---------- wiring ----------
  $("riskForm").addEventListener("submit", (e) => {
    e.preventDefault();
    const a = readForm();
    const err = $("formError");
    if (a.error) {
      err.textContent = a.error;
      err.hidden = false;
      $(a.field).focus();
      return;
    }
    err.hidden = true;
    lastInput = a;
    $("resultCard").hidden = false;
    renderResult(a);
    if (window.matchMedia("(max-width: 720px)").matches) $("resultCard").scrollIntoView({ behavior: "smooth"
});
  });

  $("riskForm").addEventListener("reset", () => {
    lastInput = null;
    $("resultCard").hidden = true;
    $("formError").hidden = true;
    $("whatIf").dataset.touched = "";
    setTimeout(() => { updateBmi(); syncSex(); syncBp(); }, 0);
  });

  ["height", "weight"].forEach((id) => $(id).addEventListener("input", updateBmi));

  function syncSex() {
    $("pregField").hidden = sex() === "male";
  }
  document.querySelectorAll('input[name="sex"]').forEach((r) => r.addEventListener("change", syncSex));

  function syncBp() {
    $("bp").disabled = $("bpUnknown").checked;
  }
  $("bpUnknown").addEventListener("change", syncBp);

  $("whatIf").addEventListener("input", () => {
    $("whatIf").dataset.touched = "1";
    updateWhatIf();
  });

  $("readingType").addEventListener("change", updateReadingFields);
  $("readingForm").addEventListener("submit", (e) => {
    e.preventDefault();
    const v = Number($("readingValue").value);
    const out = $("readingOut");
    const type = $("readingType").value;
    const limits = type === "hba1c" ? [3, 20] : $("readingUnit").value === "mmol" ? [1, 40] : [20, 700];
    if (!(v >= limits[0] && v <= limits[1])) {
      out.className = "reading-out";
      out.dataset.cat = "";
      out.textContent = t("err_range", { a: limits[0], b: limits[1] });
      return;
    }
    const cat = RISK.classifyReading(type, v, $("readingUnit").value, $("symptoms").checked);
    out.className = "reading-out " + cat;
    out.dataset.cat = cat;
    out.textContent = t("cat_" + cat);
  });

  $("langBtn").addEventListener("click", () => {
    lang = lang === "en" ? "ur" : "en";
    applyLanguage();
  });

  $("themeBtn").addEventListener("click", () => {
    const root = document.documentElement;
    root.dataset.theme = root.dataset.theme === "dark" ? "light" : "dark";
  });

  $("printBtn").addEventListener("click", () => window.print());
```

```
299
300   updateReadingFields();
301   syncSex();
302   applyLanguage();
303 })();
```

4. app/risk.js

```
/*
 * Pehchaan risk engine.
 * Pure functions, no DOM access, so the same file runs in the browser and in Node tests.
 */
(function (root) {
  "use strict";

  function sigmoid(z) {
    return 1 / (1 + Math.exp(-z));
  }

  function isMissing(v) {
    return v === null || v === undefined || Number.isNaN(v);
  }

  /**
   * Reproduces the scikit-learn pipeline: median imputation -> standard scaling -> logistic regression.
   * Each feature's contribution is coef * standardized value, i.e. how far it pushes the
   * log-odds away from an average study participant.
   */
  function score(model, input) {
    let z = model.intercept;
    const contributions = {};
    const imputed = [];
    for (const f of model.features) {
      let v = input[f];
      if (isMissing(v)) {
        v = model.medians[f];
        imputed.push(f);
      }
      const standardized = (v - model.means[f]) / model.scales[f];
      const c = model.coefficients[f] * standardized;
      contributions[f] = c;
      z += c;
    }
    const probability = sigmoid(z);
    return {
      probability: probability,
      logit: z,
      contributions: contributions,
      imputed: imputed,
      aboveThreshold: probability >= model.threshold,
    };
  }

  /** Share (0-100) of training participants whose score is below this one. */
  function percentile(model, probability) {
    const q = model.train_score_quantiles;
    let i = 0;
    while (i < q.length - 1 && q[i + 1] <= probability) i++;
    return i;
  }

  function bmi(heightCm, weightKg) {
    const m = heightCm / 100;
    return weightKg / (m * m);
  }

  const MGDL_PER_MMOL = 18.0;

  /**
   * Classifies a lab reading using American Diabetes Association diagnostic cut-offs.
   * Rule-based on purpose: this is not part of the learned model.
   * Returns one of: normal, prediabetes, diabetes, random_low, random_high_symptoms, random_high.
   */
  function classifyReading(type, value, unit, hasSymptoms) {
    if (type === "hba1c") {
      if (value >= 6.5) return "diabetes";
      if (value >= 5.7) return "prediabetes";
      return "normal";
    }
    const mgdl = unit === "mmol" ? value * MGDL_PER_MMOL : value;
    if (type === "fasting") {
```

```
      if (mgdl >= 126) return "diabetes";
      if (mgdl >= 100) return "prediabetes";
      return "normal";
    }
    if (type === "ogtt") {
      if (mgdl >= 200) return "diabetes";
      if (mgdl >= 140) return "prediabetes";
      return "normal";
    }
    if (type === "random") {
      if (mgdl < 200) return "random_low";
      return hasSymptoms ? "random_high_symptoms" : "random_high";
    }
    throw new Error("Unknown reading type: " + type);
  }

  const api = { score: score, percentile: percentile, bmi: bmi, classifyReading: classifyReading };
  if (typeof module !== "undefined" && module.exports) module.exports = api;
  else root.PehchaanRisk = api;
})(this);
```

5. app/i18n.js

```
/* Interface text in English and Urdu. Keys are shared; {x} marks a value filled in at runtime. */
window.PEHCHAAN_I18N = {
  en: {
    dir: "ltr",
    app_name: "Pehchaan",
    lang_toggle: "اردو",
    theme_toggle: "Theme",
    tagline: "Early diabetes risk screening, no lab test needed.",
    hero_body: "Answer six questions you already know the answers to. A transparent model trained on clinical data tells you whether a blood test should be your next step, and shows exactly why.",
    disclaimer: "A screening aid, not a diagnosis. Only a blood test can confirm diabetes.",
    privacy: "Runs entirely in your browser. Nothing you enter is sent or stored.",
    form_title: "About you",
    sex: "Sex",
    female: "Female",
    male: "Male",
    age: "Age (years)",
    height: "Height (cm)",
    weight: "Weight (kg)",
    bmi_label: "Body mass index",
    pregnancies: "Number of pregnancies",
    bp: "Diastolic blood pressure (the lower number, mmHg)",
    unknown: "I don't know",
    family: "Parents, brothers or sisters with diabetes",
    fam_none: "None",
    fam_one: "One",
    fam_multiple: "Two or more",
    submit: "Check my risk",
    reset: "Start over",
    result_title: "Your result",
    tier_high: "Above the screening threshold",
    tier_high_msg: "Your answers resemble people in the study who had diabetes. Get a fasting blood sugar or HbA1c test at a lab or clinic.",
    tier_low: "Below the screening threshold",
    tier_low_msg: "Your answers place you in a lower-risk group. This does not rule out diabetes. If you have symptoms such as heavy thirst or frequent urination, get tested anyway.",
    score: "Model score",
    percentile_tpl: "Higher than {p}% of study participants",
    threshold_lbl: "Screening threshold",
    why_title: "What drove this result",
    why_note: "Compared with the average study participant. Cyan bars raised your score; grey bars lowered it.",
    imputed: "not provided, study median used",
    whatif_title: "What if my weight changed?",
    whatif_note: "Shows how the model score moves with weight. It describes a pattern in the data, not a promise about your health.",
    whatif_tpl: "At {w} kg (BMI {b}), model score {s}",
    reading_title: "Already have a blood sugar result?",
    reading_note: "Checked against American Diabetes Association cut-offs. This part uses fixed clinical rules, not the AI model.",
    reading_type: "Test type",
    t_fasting: "Fasting glucose (8+ hours without food)",
    t_ogtt: "2-hour glucose after a glucose drink (OGTT)",
    t_hba1c: "HbA1c (%)",
    t_random: "Random glucose (any time of day)",
    reading_value: "Result",
    unit: "Unit",
    symptoms: "I have classic symptoms: heavy thirst, frequent urination or unexplained weight loss",
    check_reading: "Check result",
    cat_normal: "Normal range.",
    cat_prediabetes: "Prediabetes range. Talk to a doctor about follow-up testing and lifestyle changes.",
    cat_diabetes: "Diabetes range. See a doctor. A repeat test on another day is normally needed to confirm.",
    cat_random_low: "Below 200 mg/dL. A random reading on its own cannot rule diabetes in or out.",
    cat_random_high: "200 mg/dL or higher. See a doctor soon for a fasting glucose or HbA1c test.",
    cat_random_high_symptoms: "200 mg/dL or higher with classic symptoms. This meets the diagnostic criterion. See a doctor promptly.",
    how_title: "How it works",
    how_body: "A logistic regression model trained on {n} women from the Pima Indians Diabetes Database. It uses only answers you can give without a blood test. On a held-out test set it reached an AUC of {auc} (95% CI {auc_lo} to {auc_hi}), sensitivity {sens} and specificity {spec} at the screening threshold. Adding a glucose measurement raises AUC to {ref}, which is why the app points you to a blood test when your score is high.",
    limits_title: "Known limitations",
```

```
    limit_1: "Trained only on women of Pima heritage in Arizona. It has not been validated in Pakistan or South Asia, where diabetes tends to appear at a lower BMI.",
    limit_2: "{prev}% of the study population had diabetes, far above the general population, so the raw score overstates absolute risk. Treat it as a ranking, not a probability.",
    limit_3: "Family history is mapped to an approximate value of the original pedigree function.",
    limit_4: "Moderate accuracy: {miss}% of people with diabetes in the test set scored below the threshold and would have been missed.",
    print: "Print or save summary",
    warn_male: "The model was trained only on women. Results for men are less reliable.",
    warn_age: "The study included ages 21 to 81. Results outside this range are less reliable.",
    warn_bmi: "Your BMI is outside the range seen in the study (18 to 67). Results are less reliable.",
    warn_bp: "Blood pressure is outside the range seen in the study. Results are less reliable.",
    err_range: "Enter a number between {a} and {b}.",
    err_adult: "This tool is for adults (18 and over).",
    feat_Age: "Age",
    feat_BMI: "Body mass index",
    feat_Pregnancies: "Pregnancies",
    feat_BloodPressure: "Blood pressure",
    feat_DiabetesPedigreeFunction: "Family history",
    footer: "Built for UnivaBio 2026 by Muhammad Taqui. Open source. Not a medical device.",
    summary_title: "Pehchaan screening summary",
    summary_inputs: "Answers given",
  },
  ur: {
    dir: "rtl",
    app_name: "پہچان",
    lang_toggle: "English",
    theme_toggle: "تھیم",
    tagline: "لیبارٹری ٹیسٹ کے بغیر ذیابیطس کے خطرے کی ابتدائی جانچ",
    hero_body: "چھ آسان سوالوں کے جواب دیں جو آپ پہلے سے جانتے ہیں۔ طبی ڈیٹا پر تیار کیا گیا ایک شفاف ماڈل بتاتا ہے کہ کیا آپ کو خون کا ٹیسٹ کروانا چاہیے، اور اس کی وجہ بھی دکھاتا ہے۔",
    disclaimer: "یہ صرف ابتدائی جانچ ہے، تشخیص نہیں۔ ذیابیطس کی تصدیق صرف خون کے ٹیسٹ سے ہو سکتی ہے۔",
    privacy: "یہ مکمل طور پر آپ کے براؤزر میں چلتا ہے۔ آپ کی معلومات نہ کہیں بھیجی جاتی ہیں نہ محفوظ کی جاتی ہیں۔",
    form_title: "آپ کے بارے میں",
    sex: "جنس",
    female: "خاتون",
    male: "مرد",
    age: "عمر (سال)",
    height: "قد (سینٹی میٹر)",
    weight: "وزن (کلوگرام)",
    bmi_label: "باڈی ماس انڈیکس (BMI)",
    pregnancies: "حمل کی تعداد",
    bp: "ڈائسٹولک بلڈ پریشر (نیچے والا نمبر، mmHg)",
    unknown: "مجھے معلوم نہیں",
    family: "والدین یا بہن بھائیوں میں ذیابیطس",
    fam_none: "کوئی نہیں",
    fam_one: "ایک",
    fam_multiple: "دو یا زیادہ",
    submit: "میرا خطرہ جانچیں",
    reset: "دوبارہ شروع کریں",
    result_title: "آپ کا نتیجہ",
    tier_high: "اسکریننگ حد سے اوپر",
    tier_high_msg: "آپ کے جوابات تحقیق کے اُن افراد سے ملتے ہیں جنہیں ذیابیطس تھی۔ کسی لیبارٹری یا کلینک سے ٹیسٹ کروائیں۔ HbA1c فاسٹنگ شوگر یا",
    tier_low: "اسکریننگ حد سے نیچے",
    tier_low_msg: "آپ کے جوابات آپ کو کم خطرے والے گروپ میں رکھتے ہیں۔ اس سے ذیابیطس کا امکان ختم نہیں ہوتا۔ اگر شدید پیاس یا بار بار پیشاب جیسی علامات ہوں تو ٹیسٹ ضرور کروائیں۔",
    score: "ماڈل اسکور",
    percentile_tpl: "تحقیق کے {p}% شرکاء سے زیادہ",
    threshold_lbl: "اسکریننگ حد",
    why_title: "اس نتیجے کی وجوہات",
    why_note: "تحقیق کے اوسط شریک کے مقابلے میں۔ نیلی پٹیوں نے اسکور بڑھایا، سرمئی پٹیوں نے کم کیا۔",
    imputed: "فراہم نہیں کیا گیا، تحقیق کی درمیانی قدر استعمال ہوئی",
    whatif_title: "اگر میرا وزن بدل جائے تو؟",
    whatif_note: "یہ دکھاتا ہے کہ وزن کے ساتھ ماڈل اسکور کیسے بدلتا ہے۔ یہ ڈیٹا کا ایک رجحان ہے، آپ کی صحت کے بارے میں کوئی وعدہ نہیں۔",
    whatif_tpl: "{w} کلوگرام پر (BMI {b})، ماڈل اسکور {s}",
    reading_title: "کیا آپ کے پاس شوگر ٹیسٹ کا نتیجہ ہے؟",
    reading_note: "ماڈل نہیں بلکہ مقررہ طبی AI امریکن ذیابیطس ایسوسی ایشن کی حدود کے مطابق جانچا جاتا ہے۔ یہ حصہ اصول استعمال کرتا ہے۔",
    reading_type: "ٹیسٹ کی قسم",
    t_fasting: "فاسٹنگ شوگر (8 گھنٹے یا زیادہ بغیر کھائے)",
    t_ogtt: "گلوکوز پینے کے 2 گھنٹے بعد شوگر (OGTT)",
    t_hba1c: "HbA1c (%)",
    t_random: "رینڈم شوگر (دن کے کسی بھی وقت)",
```

```
    reading_value: "نتیجہ",
    unit: "اکائی",
    symptoms: "مجھے واضح علامات ہیں: شدید پیاس، بار بار پیشاب یا بلا وجہ وزن میں کمی",
    check_reading: "نتیجہ جانچیں",
    cat_normal: "نارمل حد۔",
    cat_prediabetes: "پری ذیابیطس کی حد۔ مزید ٹیسٹ اور طرزِ زندگی میں تبدیلی کے بارے میں ڈاکٹر سے بات کریں۔",
    cat_diabetes: "ذیابیطس کی حد۔ ڈاکٹر سے رجوع کریں۔ تصدیق کے لیے عام طور پر کسی دوسرے دن دوبارہ ٹیسٹ ضروری ہوتا ہے۔",
    cat_random_low: "200 mg/dL سے کم۔ صرف رینڈم ریڈنگ سے ذیابیطس کی تصدیق یا نفی نہیں ہو سکتی۔",
    cat_random_high: "200 mg/dL یا زیادہ۔ جلد ڈاکٹر سے ملیں اور فاسٹنگ شوگر یا HbA1c ٹیسٹ کروائیں۔",
    cat_random_high_symptoms: "200 mg/dL یا زیادہ۔ یہ تشخیصی معیار پر پورا اترتا ہے۔ فوراً واضح علامات کے ساتھ ڈاکٹر سے رجوع کریں۔",
    how_title: "یہ کیسے کام کرتا ہے",
    how_body: "یہ لاجسٹک ریگریشن ماڈل Pima Indians Diabetes Database کی {n} خواتین کے ڈیٹا پر تیار کیا گیا ہے اور صرف وہ معلومات استعمال کرتا ہے جو خون کے ٹیسٹ کے بغیر دی جا سکیں۔ الگ رکھے گئے ٹیسٹ ڈیٹا پر اس کا AUC {auc} رہا ۔ AUC شوگر کی پیمائش شامل کرنے سے{spec} اور مخصوصیت {sens} اسکریننگ حد پر حساسیت ،(95% CI: {auc_lo} تا {auc_hi}) ہو جاتا ہے، اسی لیے زیادہ اسکور پر ایپ آپ کو خون کے ٹیسٹ کا مشورہ دیتی ہے۔ {ref}",
    limits_title: "معلوم حدود",
    limit_1: "یہ ماڈل صرف ایریزونا کی Pima نسل کی خواتین کے ڈیٹا پر تیار ہوا ہے۔ پاکستان یا جنوبی ایشیا میں اس پر بھی ہو جاتی ہے۔ BMI کی جانچ نہیں ہوئی، جہاں ذیابیطس عموماً کم",
    limit_2: "تحقیق میں شامل {prev}% افراد کو ذیابیطس تھی، جو عام آبادی سے کہیں زیادہ ہے، اس لیے اسکور اصل خطرے کو بڑھا کر دکھاتا ہے۔ اسے درجہ بندی سمجھیں، امکان نہیں۔",
    limit_3: "خاندانی پس منظر کو اصل pedigree function کی تخمینی قدر میں بدلا گیا ہے۔",
    limit_4: "ٹیسٹ ڈیٹا میں ذیابیطس والے {miss}% درستگی درمیانی ہے: اس سے افراد کا اسکور حد سے نیچے رہا، یعنی وہ جانچ سے رہ جاتے۔",
    print: "خلاصہ پرنٹ یا محفوظ کریں",
    warn_male: "ماڈل صرف خواتین کے ڈیٹا پر تیار کیا گیا ہے۔ مردوں کے لیے نتائج کم قابلِ اعتماد ہیں۔",
    warn_age: "تحقیق میں 21 سے 81 سال کے افراد شامل تھے۔ اس حد سے باہر نتائج کم قابلِ اعتماد ہیں۔",
    warn_bmi: "آپ کا BMI تحقیق کی حد (18 سے 67) سے باہر ہے۔ نتائج کم قابلِ اعتماد ہیں۔",
    warn_bp: "بلڈ پریشر تحقیق کی حد سے باہر ہے۔ نتائج کم قابلِ اعتماد ہیں۔",
    err_range: "{a} اور {b} کے درمیان عدد درج کریں۔",
    err_adult: "یہ ٹول صرف بالغ افراد (18 سال یا زیادہ) کے لیے ہے۔",
    feat_Age: "عمر",
    feat_BMI: "باڈی ماس انڈیکس",
    feat_Pregnancies: "حمل",
    feat_BloodPressure: "بلڈ پریشر",
    feat_DiabetesPedigreeFunction: "خاندانی پس منظر",
    footer: "UnivaBio 2026 کے لیے محمد تقی نے تیار کیا۔ اوپن سورس طبی آلہ نہیں۔",
    summary_title: "پہچان اسکریننگ خلاصہ",
    summary_inputs: "دیے گئے جوابات",
  },
};
```

6. ml/train.py

```python
"""Train Pehchaan's no-lab diabetes risk model and export it for the browser.

Pipeline
  1. Load the Pima Indians Diabetes dataset (768 women, age 21+).
  2. Treat physiologically impossible zeros as missing values.
  3. Hold out 25% as a test set that is touched exactly once.
  4. Compare candidate models with 5-fold cross-validation on the training set.
  5. Pick the screening threshold on cross-validated training predictions
     (target sensitivity >= 0.80), never on the test set.
  6. Fit the final no-lab logistic regression, evaluate on the test set with
     bootstrap confidence intervals, and export everything the browser needs.

Run:  python ml/train.py
"""
from __future__ import annotations

import json
import pathlib

import numpy as np
import pandas as pd
from sklearn.calibration import calibration_curve
from sklearn.ensemble import HistGradientBoostingClassifier, RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import brier_score_loss, roc_auc_score, roc_curve
from sklearn.model_selection import StratifiedKFold, cross_val_predict, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

SEED = 42
ROOT = pathlib.Path(__file__).resolve().parents[1]
DATA = ROOT / "ml" / "data" / "pima.csv"
REPORTS = ROOT / "ml" / "reports"
APP = ROOT / "app"
TESTS = ROOT / "tests"

COLUMNS = [
    "Pregnancies", "Glucose", "BloodPressure", "SkinThickness",
    "Insulin", "BMI", "DiabetesPedigreeFunction", "Age", "Outcome",
]
# Features a person can answer at home, without any blood test.
NO_LAB = ["Pregnancies", "BloodPressure", "BMI", "DiabetesPedigreeFunction", "Age"]
# All eight features, used only as a reference point in the report.
FULL = NO_LAB + ["Glucose", "SkinThickness", "Insulin"]
# A recorded value of 0 is impossible for these, so it means "not measured".
ZERO_IS_MISSING = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
TARGET_SENSITIVITY = 0.80


def load() -> pd.DataFrame:
    df = pd.read_csv(DATA, header=None, names=COLUMNS)
    df[ZERO_IS_MISSING] = df[ZERO_IS_MISSING].replace(0, np.nan)
    return df


def logreg() -> Pipeline:
    return Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),
        ("model", LogisticRegression(max_iter=1000)),
    ])


def candidates() -> dict[str, tuple[Pipeline, list[str]]]:
    rf = Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("model", RandomForestClassifier(n_estimators=400, min_samples_leaf=5, random_state=SEED)),
    ])
    gb = Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("model", HistGradientBoostingClassifier(max_depth=3, learning_rate=0.05, random_state=SEED)),
    ])
```

```
    return {
        "Logistic regression, no-lab (deployed)": (logreg(), NO_LAB),
        "Random forest, no-lab": (rf, NO_LAB),
        "Gradient boosting, no-lab": (gb, NO_LAB),
        "Logistic regression, all 8 features incl. glucose (reference)": (logreg(), FULL),
    }


def sens_spec(y: np.ndarray, p: np.ndarray, thr: float) -> tuple[float, float]:
    pred = p >= thr
    sens = (pred & (y == 1)).sum() / (y == 1).sum()
    spec = (~pred & (y == 0)).sum() / (y == 0).sum()
    return float(sens), float(spec)


def pick_threshold(y: np.ndarray, p: np.ndarray, target: float) -> float:
    """Highest threshold whose sensitivity still meets the target."""
    for thr in np.round(np.arange(0.95, 0.0, -0.005), 3):
        if sens_spec(y, p, thr)[0] >= target:
            return float(thr)
    return 0.0


def bootstrap_ci(y, p, thr, n=2000):
    rng = np.random.default_rng(SEED)
    stats = {"auc": [], "sensitivity": [], "specificity": []}
    for _ in range(n):
        idx = rng.integers(0, len(y), len(y))
        yb, pb = y[idx], p[idx]
        if yb.min() == yb.max():
            continue
        stats["auc"].append(roc_auc_score(yb, pb))
        s, sp = sens_spec(yb, pb, thr)
        stats["sensitivity"].append(s)
        stats["specificity"].append(sp)
    return {k: [float(np.percentile(v, 2.5)), float(np.percentile(v, 97.5))] for k, v in stats.items()}


def main() -> None:
    REPORTS.mkdir(parents=True, exist_ok=True)
    df = load()
    train, test = train_test_split(df, test_size=0.25, stratify=df["Outcome"], random_state=SEED)
    y_tr, y_te = train["Outcome"].to_numpy(), test["Outcome"].to_numpy()
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)

    # Step 4: model comparison on training folds only.
    comparison = []
    for name, (pipe, feats) in candidates().items():
        p = cross_val_predict(pipe, train[feats], y_tr, cv=cv, method="predict_proba")[:, 1]
        comparison.append({"model": name, "cv_auc": round(float(roc_auc_score(y_tr, p)), 3),
                           "cv_brier": round(float(brier_score_loss(y_tr, p)), 3)})
        print(f"{name:<66} CV AUC {comparison[-1]['cv_auc']:.3f}")

    # Step 5: threshold from cross-validated training predictions.
    final = logreg()
    p_cv = cross_val_predict(final, train[NO_LAB], y_tr, cv=cv, method="predict_proba")[:, 1]
    thr = pick_threshold(y_tr, p_cv, TARGET_SENSITIVITY)

    # Step 6: fit once, test once.
    final.fit(train[NO_LAB], y_tr)
    p_te = final.predict_proba(test[NO_LAB])[:, 1]
    sens, spec = sens_spec(y_te, p_te, thr)
    test_metrics = {
        "auc": round(float(roc_auc_score(y_te, p_te)), 3),
        "brier": round(float(brier_score_loss(y_te, p_te)), 3),
        "sensitivity": round(sens, 3),
        "specificity": round(spec, 3),
        "ci95": {k: [round(a, 3), round(b, 3)] for k, (a, b) in bootstrap_ci(y_te, p_te, thr).items()},
        "n_test": int(len(y_te)),
        "positives_test": int(y_te.sum()),
    }
    full_ref = logreg().fit(train[FULL], y_tr)
    ref_auc = round(float(roc_auc_score(y_te, full_ref.predict_proba(test[FULL])[:, 1])), 3)

    imp, sc, lr = final.named_steps["impute"], final.named_steps["scale"], final.named_steps["model"]
    p_train = final.predict_proba(train[NO_LAB])[:, 1]
```

```
    dpf = train["DiabetesPedigreeFunction"]

    model = {
        "name": "Pehchaan no-lab diabetes risk model",
        "version": "1.0.0",
        "features": NO_LAB,
        "medians": dict(zip(NO_LAB, map(float, imp.statistics_))),
        "means": dict(zip(NO_LAB, map(float, sc.mean_))),
        "scales": dict(zip(NO_LAB, map(float, sc.scale_))),
        "coefficients": dict(zip(NO_LAB, map(float, lr.coef_[0]))),
        "intercept": float(lr.intercept_[0]),
        "threshold": thr,
        "threshold_rule": f"Highest cut-off reaching sensitivity >= {TARGET_SENSITIVITY} on 5-fold CV of the
training set",
        # Score distribution of the training population, for percentile display.
        "train_score_quantiles": [float(q) for q in np.quantile(p_train, np.linspace(0, 1, 101))],
        # Approximate mapping from a family-history question to the pedigree function.
        "family_history_dpf": {
            "none": float(dpf.quantile(0.25)),
            "one": float(dpf.quantile(0.50)),
            "multiple": float(dpf.quantile(0.75)),
        },
        "training_ranges": {f: [float(train[f].min()), float(train[f].max())] for f in NO_LAB},
        "dataset": {
            "name": "Pima Indians Diabetes Database (NIDDK, via UCI)",
            "n_total": int(len(df)),
            "prevalence": round(float(df["Outcome"].mean()), 3),
            "population": "Women of Pima heritage, age 21+, Arizona, USA",
        },
        "test_metrics": test_metrics,
        "reference_auc_with_glucose": ref_auc,
        "cv_comparison": comparison,
    }
    (APP / "model.json").write_text(json.dumps(model, indent=2))
    # Same data as a script, so the app also works when opened straight from disk.
    (APP / "model.js").write_text("window.PEHCHAAN_MODEL = " + json.dumps(model) + ";\n")
    (REPORTS / "metrics.json").write_text(json.dumps(
        {"test_metrics": test_metrics, "threshold": thr,
         "reference_auc_with_glucose": ref_auc, "cv_comparison": comparison}, indent=2))

    # Parity vectors: the browser must reproduce these probabilities exactly.
    sample = test[NO_LAB].head(25)
    vectors = [{"input": {k: (None if pd.isna(v) else float(v)) for k, v in row.items()},
                "expected": float(prob)}
               for (_, row), prob in zip(sample.iterrows(), final.predict_proba(sample)[:, 1])]
    (TESTS / "parity_vectors.json").write_text(json.dumps(vectors, indent=2))

    plot(y_te, p_te, thr, sens, test_metrics["auc"], full_ref.predict_proba(test[FULL])[:, 1], ref_auc)
    print(json.dumps(test_metrics, indent=2))
    print("threshold", thr, "| reference AUC with glucose", ref_auc)


def plot(y, p, thr, sens, auc, p_ref, ref_auc) -> None:
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    navy, cyan, grey = "#0B1F3B", "#00A3D9", "#9AA4B2"
    fig, axes = plt.subplots(1, 2, figsize=(10, 4.2))

    fpr, tpr, _ = roc_curve(y, p)
    fr, tr_, _ = roc_curve(y, p_ref)
    ax = axes[0]
    ax.plot(fpr, tpr, color=navy, lw=2, label=f"No-lab model (AUC {auc:.2f})")
    ax.plot(fr, tr_, color=cyan, lw=1.5, ls="--", label=f"With glucose, reference (AUC {ref_auc:.2f})")
    ax.plot([0, 1], [0, 1], color=grey, lw=1, ls=":")
    s, sp = sens_spec(y, p, thr)
    ax.scatter([1 - sp], [s], color=navy, zorder=3, label=f"Screening cut-off {thr:.3f}")
    ax.set(xlabel="False positive rate", ylabel="True positive rate", title="ROC, held-out test set")
    ax.legend(loc="lower right", fontsize=8, frameon=False)

    ax = axes[1]
    frac, mean = calibration_curve(y, p, n_bins=6, strategy="quantile")
    ax.plot(mean, frac, marker="o", color=navy, lw=2, label="No-lab model")
    ax.plot([0, 1], [0, 1], color=grey, lw=1, ls=":", label="Perfect calibration")
    ax.set(xlabel="Predicted probability", ylabel="Observed fraction with diabetes",
```

```
          title="Calibration, held-out test set")
    ax.legend(loc="upper left", fontsize=8, frameon=False)
    for a in axes:
        a.spines[["top", "right"]].set_visible(False)
    fig.tight_layout()
    fig.savefig(REPORTS / "evaluation.png", dpi=160)
    fig.savefig(APP / "evaluation.png", dpi=110)


if __name__ == "__main__":
    main()
```

7. tests/test_risk.js

```
// Run with: node --test tests/
// 1. The browser engine must reproduce scikit-learn's probabilities.
// 2. The rule-based lab check must follow ADA cut-offs at every boundary.
const test = require("node:test");
const assert = require("node:assert/strict");
const path = require("node:path");

const risk = require(path.join(__dirname, "..", "app", "risk.js"));
const model = require(path.join(__dirname, "..", "app", "model.json"));
const vectors = require(path.join(__dirname, "parity_vectors.json"));

test("JS engine matches scikit-learn on held-out rows", () => {
  for (const v of vectors) {
    const got = risk.score(model, v.input).probability;
    assert.ok(Math.abs(got - v.expected) < 1e-9, `expected ${v.expected}, got ${got}`);
  }
});

test("missing blood pressure is imputed with the training median", () => {
  const base = { Pregnancies: 2, BMI: 30, DiabetesPedigreeFunction: 0.4, Age: 40 };
  const a = risk.score(model, { ...base, BloodPressure: null });
  const b = risk.score(model, { ...base, BloodPressure: model.medians.BloodPressure });
  assert.equal(a.probability, b.probability);
  assert.deepEqual(a.imputed, ["BloodPressure"]);
});

test("contributions sum to the logit", () => {
  const r = risk.score(model, { Pregnancies: 4, BloodPressure: 80, BMI: 35, DiabetesPedigreeFunction: 0.6,
Age: 50 });
  const sum = Object.values(r.contributions).reduce((s, x) => s + x, model.intercept);
  assert.ok(Math.abs(sum - r.logit) < 1e-12);
});

test("higher BMI never lowers the score", () => {
  const base = { Pregnancies: 1, BloodPressure: 70, DiabetesPedigreeFunction: 0.3, Age: 35 };
  let prev = 0;
  for (let bmi = 18; bmi <= 60; bmi += 2) {
    const p = risk.score(model, { ...base, BMI: bmi }).probability;
    assert.ok(p >= prev);
    prev = p;
  }
});

test("percentile stays within 0..100", () => {
  assert.equal(risk.percentile(model, 0), 0);
  assert.equal(risk.percentile(model, 1), 100);
});

test("BMI formula", () => {
  assert.ok(Math.abs(risk.bmi(170, 72.25) - 25) < 1e-9);
});

test("ADA cut-offs at their boundaries", () => {
  const c = risk.classifyReading;
  assert.equal(c("fasting", 99, "mgdl"), "normal");
  assert.equal(c("fasting", 100, "mgdl"), "prediabetes");
  assert.equal(c("fasting", 125, "mgdl"), "prediabetes");
  assert.equal(c("fasting", 126, "mgdl"), "diabetes");
  assert.equal(c("fasting", 7.0, "mmol"), "diabetes");
  assert.equal(c("ogtt", 139, "mgdl"), "normal");
  assert.equal(c("ogtt", 140, "mgdl"), "prediabetes");
  assert.equal(c("ogtt", 200, "mgdl"), "diabetes");
  assert.equal(c("hba1c", 5.6), "normal");
  assert.equal(c("hba1c", 5.7), "prediabetes");
  assert.equal(c("hba1c", 6.5), "diabetes");
  assert.equal(c("random", 180, "mgdl", true), "random_low");
  assert.equal(c("random", 210, "mgdl", false), "random_high");
  assert.equal(c("random", 210, "mgdl", true), "random_high_symptoms");
});
```

8. scripts/build_docs.py

```
"""Build the two PDFs Devpost asks for.

  docs/Pehchaan_Project_Description.pdf   one page: purpose and key features
  docs/Pehchaan_Code.pdf                  every source file written for the project

Needs: pip install playwright pygments && python -m playwright install chromium
Run:   python scripts/build_docs.py
"""
import base64
import html
import json
import pathlib

from playwright.sync_api import sync_playwright
from pygments import highlight
from pygments.formatters import HtmlFormatter
from pygments.lexers import get_lexer_for_filename

ROOT = pathlib.Path(__file__).resolve().parents[1]
DOCS = ROOT / "docs"
CODE_FILES = [
    "app/index.html", "app/styles.css", "app/app.js", "app/risk.js", "app/i18n.js",
    "ml/train.py", "tests/test_risk.js", "scripts/build_docs.py",
    "package.json", "requirements.txt",
]

BASE_CSS = """
@page { size: A4; margin: 14mm 15mm; }
body { font-family: Inter, 'Segoe UI', sans-serif; color: #111827; font-size: 10pt; line-height: 1.45;
margin: 0; }
h1 { font-size: 22pt; margin: 0; color: #0B1F3B; letter-spacing: -0.01em; }
h2 { font-size: 11pt; margin: 12px 0 4px; color: #0B1F3B; }
.sub { color: #00799F; font-weight: 600; margin: 2px 0 10px; }
p { margin: 0 0 6px; }
ul { margin: 0 0 6px; padding-left: 16px; } li { margin-bottom: 2px; }
.cols { display: grid; grid-template-columns: 1.15fr 1fr; gap: 16px; }
table { border-collapse: collapse; width: 100%; font-size: 9pt; }
td, th { border-bottom: 1px solid #DCE3EC; padding: 3px 4px; text-align: left; }
th { color: #5B6676; font-weight: 600; }
img { width: 100%; border: 1px solid #DCE3EC; border-radius: 4px; }
.bar { border-left: 3px solid #00C2FF; padding: 4px 8px; background: #F2F8FC; margin: 8px 0; }
.foot { color: #5B6676; font-size: 8pt; margin-top: 8px; }
"""


def description_html() -> str:
    m = json.loads((ROOT / "app" / "model.json").read_text())
    t = m["test_metrics"]
    img = base64.b64encode((ROOT / "ml" / "reports" / "evaluation.png").read_bytes()).decode()
    cv = {c["model"].split(",")[0] + (" + glucose" if "glucose" in c["model"] else ""): c["cv_auc"]
          for c in m["cv_comparison"]}
    cv_rows = "".join(f"<tr><td>{html.escape(k)}</td><td>{v:.3f}</td></tr>" for k, v in cv.items())
    return f"""<!doctype html><html><head><meta charset="utf-8"><style>{BASE_CSS}</style></head><body>
<h1>Pehchaan</h1>
<p class="sub">Early diabetes risk screening without a lab test. UnivaBio 2026, theme: AI for Human Health.
Muhammad Taqui.</p>

<div class="bar"><b>Purpose.</b> Many people with type 2 diabetes do not know they have it, because testing
needs a lab visit.
Pehchaan answers one question from information a person already knows: should I get a blood test now?
It explains every result, works in English and Urdu, and runs fully in the browser with no data sent
anywhere.</div>

<div class="cols"><div>
<h2>Key features</h2>
<ul>
<li><b>Six-question screen.</b> Sex, age, height and weight (live BMI), pregnancies, optional blood pressure,
family history.</li>
<li><b>Clear outcome.</b> Above or below a screening threshold, percentile among study participants, and a
specific next step.</li>
<li><b>Exact explanation.</b> A bar per factor showing its contribution to the score, compared with an
average participant.</li>
<li><b>What-if slider.</b> How the score moves with weight, labelled as a data pattern rather than a promise.
```

```
</li>
<li><b>Lab result checker.</b> Fasting, OGTT, HbA1c or random glucose classified by ADA cut-offs; rule-based and separate from the model.</li>
<li><b>Accessible.</b> Full Urdu interface with right-to-left layout, dark and light themes, mobile layout, printable clinic summary.</li>
<li><b>Private and offline.</b> Static files only; no server, no account, no storage.</li>
<li><b>Model card in the app.</b> Data source, metrics with confidence intervals, plots and four stated limitations.</li>
</ul>
<h2>How it works</h2>
<p>Logistic regression on the Pima Indians Diabetes Database ({m['dataset']['n_total']} women), using five features answerable without a blood test.
Impossible zeros treated as missing and median-imputed. 25% held out; model choice and threshold
(sensitivity target 0.80) fixed by 5-fold cross-validation on the training set; test set used once.
The exported model runs in JavaScript, and tests confirm it matches scikit-learn to 1e-9.</p>
</div><div>
<h2>Held-out test results (n = {t['n_test']}, {t['positives_test']} with diabetes)</h2>
<table><tr><th>Metric</th><th>Value</th><th>95% CI</th></tr>
<tr><td>AUC</td><td>{t['auc']:.3f}</td><td>{t['ci95']['auc'][0]:.2f} to {t['ci95']['auc'][1]:.2f}</td></tr>
<tr><td>Sensitivity at {m['threshold']}</td><td>{t['sensitivity']:.3f}</td><td>{t['ci95']['sensitivity'][0]:.2f} to {t['ci95']['sensitivity'][1]:.2f}</td></tr>
<tr><td>Specificity at {m['threshold']}</td><td>{t['specificity']:.3f}</td><td>{t['ci95']['specificity'][0]:.2f} to {t['ci95']['specificity'][1]:.2f}</td></tr>
</table>
<h2>Cross-validated AUC, training set</h2>
<table>{cv_rows}</table>
<p style="margin-top:6px">Tree ensembles gave no gain, so the explainable model was kept. Adding glucose lifts test AUC to {m['reference_auc_with_glucose']:.2f}, which is why a high score leads to a blood test.</p>
<img src="{img}">
</div></div>

<h2>Limitations, stated in the app</h2>
<p>Trained only on women of Pima heritage in Arizona; not validated in South Asia, and men receive a warning.
Study prevalence was {m['dataset']['prevalence']*100:.0f}%, so raw scores overstate absolute risk. Family history is an approximate mapping.
{round((1-t['sensitivity'])*100)}% of test-set cases fell below the threshold. Not a medical device.</p>
<p><b>Finished:</b> everything above. <b>Planned:</b> retraining on South Asian data with waist circumference, clinic validation, Android app.</p>
<p class="foot">Data: Pima Indians Diabetes Database (NIDDK, via the UCI Machine Learning Repository). Smith et al., Proc. Symp. Computer Applications in Medical Care, 1988, 261 to 265. Code: MIT licence.</p>
</body></html>"""


def code_html() -> str:
    fmt = HtmlFormatter(style="friendly", linenos="inline", nobackground=True)
    parts = []
    toc = []
    for i, rel in enumerate(CODE_FILES, 1):
        src = (ROOT / rel).read_text(encoding="utf-8")
        lines = src.count("\n") + 1
        toc.append(f"<li>{rel} ({lines} lines)</li>")
        try:
            body = highlight(src, get_lexer_for_filename(rel), fmt)
        except Exception:
            body = f"<pre>{html.escape(src)}</pre>"
        parts.append(f'<section><h2>{i}. {rel}</h2>{body}</section>')
    css = BASE_CSS + fmt.get_style_defs(".highlight") + """
section { page-break-before: always; }
pre { font-family: 'JetBrains Mono', Menlo, monospace; font-size: 7.4pt; line-height: 1.35;
      white-space: pre-wrap; word-break: break-word; margin: 0; }
.highlight .linenos { color: #9AA4B2; padding-right: 6px; }
"""
    return f"""<!doctype html><html><head><meta charset="utf-8"><style>{css}</style></head><body>
<h1>Pehchaan: source code</h1>
<p class="sub">UnivaBio 2026 submission. Muhammad Taqui. MIT licence.</p>
<p>All code written for the project, in the order a reader would follow it: the web app, the risk engine,
the interface text, the training pipeline, the tests and the documentation build script.
Generated files (model.json, model.js, parity_vectors.json) and the dataset are not reprinted;
they are produced by ml/train.py.</p>
<h2>Contents</h2><ol>{''.join(toc)}</ol>
{''.join(parts)}
</body></html>"""


def render(doc: str, out: pathlib.Path, page_numbers: bool) -> None:
    with sync_playwright() as p:
```

```
        browser = p.chromium.launch()
        page = browser.new_page()
        page.set_content(doc, wait_until="load")
        footer = ('<div style="font-size:8px;width:100%;text-align:center;color:#888">'
                  'Pehchaan source code, page <span class="pageNumber"></span> of <span class="totalPages">
</span></div>')
        page.pdf(path=str(out), format="A4", print_background=True, prefer_css_page_size=True,
                 display_header_footer=page_numbers, header_template="<div></div>", footer_template=footer)
        browser.close()


if __name__ == "__main__":
    DOCS.mkdir(exist_ok=True)
    render(description_html(), DOCS / "Pehchaan_Project_Description.pdf", page_numbers=False)
    render(code_html(), DOCS / "Pehchaan_Code.pdf", page_numbers=True)
    print("PDFs written to", DOCS)
```

9. package.json

```
{
  "name": "pehchaan",
  "version": "1.0.0",
  "private": true,
  "description": "Early diabetes risk screening without a lab test. Explainable, bilingual, runs in the browser.",
  "scripts": {
    "test": "node --test tests/test_risk.js",
    "serve": "python3 -m http.server 8000 --directory app"
  },
  "license": "MIT"
}
```

10. requirements.txt

```
numpy
pandas
scikit-learn>=1.3
matplotlib
```